JS Problems SDE1

11 min read Updated 20 May 2026 Topic 48 of 77

Bookmark Mark done

JS Problems — SDE1 Level

Problems: 20 **Companies:** Flipkart, Swiggy, Razorpay, Amazon, Cred, PhonePe, Atlassian, Walmart

17.1 Implement `Array.prototype.map`

JAVASCRIPT Copy

```
Array.prototype.myMap = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) { // Handle sparse arrays — skip holes
      result[i] = callback.call(thisArg, this[i], i, this);
    }
  }
  return result;
};

// Test
[1, 2, 3].myMap(x => x * 2); // [2, 4, 6]
[1, , 3].myMap(x => x * 2);  // [2, empty, 6] — preserves holes
```

What interviewers check: `thisArg` support, sparse array handling (`i in this`), passing index and array as arguments.

JAVASCRIPT Copy

```
Array.prototype.myFilter = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this && callback.call(thisArg, this[i], i, this)) {
      result.push(this[i]);
    }
  }
  return result;
};

[1, 2, 3, 4, 5].myFilter(x => x % 2 === 0); // [2, 4]
```

17.3 Implement `Array.prototype.reduce`

JAVASCRIPT Copy

```
Array.prototype.myReduce = function(callback, initialValue) {
  let accumulator;
  let startIndex;

  if (initialValue !== undefined) {
    accumulator = initialValue;
    startIndex = 0;
  } else {
    if (this.length === 0) throw new TypeError('Reduce of empty array with
    // Find first non-hole element
    let found = false;
    for (let i = 0; i < this.length; i++) {
      if (i in this) { accumulator = this[i]; startIndex = i + 1; found =
    }
    if (!found) throw new TypeError('Reduce of empty array with no initial
  }

  for (let i = startIndex; i < this.length; i++) {
    if (i in this) {
```

```
    accumulator = callback(accumulator, this[i], i, this);
  }
  return accumulator;
};

[1, 2, 3].myReduce((acc, val) => acc + val, 0);  // 6
[1, 2, 3].myReduce((acc, val) => acc + val);      // 6 (first element as i
```

What interviewers check: Handling missing `initialValue` (use first element), empty array with no initial value (throw TypeError), sparse arrays.

17.4 Implement `Function.prototype.bind`

JAVASCRIPT Copy

```
Function.prototype.myBind = function(thisArg, ...boundArgs) {
  const fn = this;

  return function(...callArgs) {
    return fn.apply(thisArg, [...boundArgs, ...callArgs]);
  };
};

// Test
const greet = function(greeting, name) {
  return `${greeting}, ${name}! I'm ${this.title}`;
};

const boundGreet = greet.myBind({ title: 'Engineer' }, 'Hello');
boundGreet('Ankit'); // "Hello, Ankit! I'm Engineer"
```

What interviewers check: Partial application (pre-filling some arguments), `this` binding, combining bound args with call-time args.

Advanced follow-up: Handle `new` operator — bound function used as constructor should ignore the bound `this`:

```
Function.prototype.myBind = function(thisArg, ...boundArgs) {
  const fn = this;
  const bound = function(...callArgs) {
    const context = this instanceof bound ? this : thisArg;
    return fn.apply(context, [...boundArgs, ...callArgs]);
  };
  bound.prototype = Object.create(fn.prototype);
  return bound;
};
```

17.5 Implement `Function.prototype.call` and `apply`

JAVASCRIPT Copy

```
Function.prototype.myCall = function(thisArg, ...args) {
  thisArg = thisArg ?? globalThis;
  thisArg = Object(thisArg); // Handle primitives
  const key = Symbol();      // Unique key to avoid property collision
  thisArg[key] = this;
  const result = thisArg[key](...args);
  delete thisArg[key];
  return result;
};

Function.prototype.myApply = function(thisArg, args = []) {
  return this.myCall(thisArg, ...args);
};

// Test
function greet(greeting) { return `${greeting}, ${this.name}`; }
greet.myCall({ name: 'Ankit' }, 'Hello'); // "Hello, Ankit"
greet.myApply({ name: 'Ankit' }, ['Hi']); // "Hi, Ankit"
```

Key insight: Temporarily attach the function as a method of `thisArg`, call it (so `this` refers to `thisArg`), then clean up. Use `Symbol()` to avoid overwriting existing properties.

17.6 Implement `debounce`

JAVASCRIPT Copy

```
function debounce(fn, delay) {
  let timerId;

  return function(...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// Usage
const search = debounce((query) => {
  fetch(`/api/search?q=${query}`);
}, 300);

input.addEventListener('input', (e) => search(e.target.value));
// Only fires 300ms after the LAST keystroke
```

With leading + trailing option:

JAVASCRIPT Copy

```
function debounce(fn, delay, { leading = false, trailing = true } = {}) {
  let timerId;
  let lastArgs;

  return function(...args) {
    const isFirst = !timerId && leading;

    clearTimeout(timerId);
    lastArgs = args;

    if (isFirst) fn.apply(this, args);

    timerId = setTimeout(() => {
```

```
    }, delay);
  };
}
```

17.7 Implement `throttle`

JAVASCRIPT Copy

```
function throttle(fn, interval) {
  let lastTime = 0;
  let timerId;

  return function(...args) {
    const now = Date.now();
    const remaining = interval - (now - lastTime);

    if (remaining ≤ 0) {
      clearTimeout(timerId);
      lastTime = now;
      fn.apply(this, args);
    } else if (!timerId) {
      timerId = setTimeout(() ⇒ {
        lastTime = Date.now();
        timerId = null;
        fn.apply(this, args);
      }, remaining);
    }
  };
}

// Usage — fires AT MOST once per 200ms
window.addEventListener('scroll', throttle(handleScroll, 200));
```

Debounce vs Throttle: Debounce waits for silence (fires after delay with no calls). Throttle ensures regular firing (fires at most once per interval). Debounce for search input. Throttle for scroll/resize handlers.

17.8 Implement `deepClone`

JAVASCRIPT

Copy

```
function deepClone(value) {
  // Handle primitives and null
  if (value === null || typeof value !== 'object') return value;

  // Handle Date
  if (value instanceof Date) return new Date(value.getTime());

  // Handle RegExp
  if (value instanceof RegExp) return new RegExp(value.source, value.flags

  // Handle Map
  if (value instanceof Map) {
    const clone = new Map();
    value.forEach((v, k) => clone.set(deepClone(k), deepClone(v)));
    return clone;
  }

  // Handle Set
  if (value instanceof Set) {
    const clone = new Set();
    value.forEach(v => clone.add(deepClone(v)));
    return clone;
  }

  // Handle Array and Object
  const clone = Array.isArray(value) ? [] : {};
  for (const key of Object.keys(value)) {
    clone[key] = deepClone(value[key]);
  }
  return clone;
}
```

Production shortcut: `structuredClone(value)` — handles Date, Map, Set, ArrayBuffer, circular references. Doesn't handle functions, DOM nodes, or prototypes. Available in all modern browsers and Node 17+.

Interview follow-up: "How do you handle circular references?" Use a `WeakMap` to track cloned objects.

JAVASCRIPT Copy

```javascript
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;
  if (seen.has(value)) return seen.get(value); // Circular reference

  const clone = Array.isArray(value) ? [] : {};
  seen.set(value, clone);

  for (const key of Object.keys(value)) {
    clone[key] = deepClone(value[key], seen);
  }
  return clone;
}
```

17.9 Implement `flatten` (Array)

JAVASCRIPT Copy

```javascript
// Recursive — flatten to any depth
function flatten(arr, depth = Infinity) {
  const result = [];

  for (const item of arr) {
    if (Array.isArray(item) && depth > 0) {
      result.push(...flatten(item, depth - 1));
    } else {
      result.push(item);
    }
  }
  return result;
}

// Test
flatten([1, [2, [3, [4]]]], 1);       // [1, 2, [3, [4]]]
```

Iterative version (avoids stack overflow on very deep nesting):

JAVASCRIPT Copy

```
function flattenIterative(arr) {
  const stack = [...arr];
  const result = [];
  while (stack.length) {
    const item = stack.pop();
    if (Array.isArray(item)) stack.push(...item);
    else result.push(item);
  }
  return result.reverse();
}
```

17.10 Implement `Promise`

JAVASCRIPT Copy

```
class MyPromise {
  #state = 'pending';
  #value = undefined;
  #handlers = [];

  constructor(executor) {
    const resolve = (value) => {
      if (this.#state !== 'pending') return;
      this.#state = 'fulfilled';
      this.#value = value;
      this.#handlers.forEach(h => h.onFulfilled(value));
    };

    const reject = (reason) => {
      if (this.#state !== 'pending') return;
      this.#state = 'rejected';
      this.#value = reason;
```

```
    try {
      executor(resolve, reject);
    } catch (error) {
      reject(error);
    }
  }

  then(onFulfilled, onRejected) {
    return new MyPromise((resolve, reject) => {
      const handle = () => {
        try {
          if (this.#state === 'fulfilled') {
            const result = onFulfilled ? onFulfilled(this.#value) : this.#
            resolve(result);
          } else if (this.#state === 'rejected') {
            if (onRejected) resolve(onRejected(this.#value));
            else reject(this.#value);
          }
        } catch (error) {
          reject(error);
        }
      };

      if (this.#state === 'pending') {
        this.#handlers.push({ onFulfilled: () => handle(), onRejected: ()
      } else {
        queueMicrotask(handle); // Async resolution
      }
    });
  }

  catch(onRejected) { return this.then(null, onRejected); }

  static resolve(value) { return new MyPromise(resolve => resolve(value));
  static reject(reason) { return new MyPromise((_, reject) => reject(reaso
}
```

Note: A fully spec-compliant Promise implementation is 200+ lines. In interviews, the above covers the core behavior. Key things interviewers check: state transitions

17.11 Implement `Promise.all`

JAVASCRIPT

Copy

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let completed = 0;
    const items = Array.from(promises);

    if (items.length === 0) { resolve([]); return; }

    items.forEach((promise, index) => {
      Promise.resolve(promise).then(
        (value) => {
          results[index] = value; // Maintain order
          completed++;
          if (completed === items.length) resolve(results);
        },
        reject // First rejection rejects the whole thing
      );
    });
  });
}

// Test
promiseAll([
  Promise.resolve(1),
  new Promise(r => setTimeout(() => r(2), 100)),
  Promise.resolve(3),
]).then(console.log); // [1, 2, 3] — order preserved
```

What interviewers check: Order preservation (`results[index]` , not `results.push`), handling non-Promise values (`Promise.resolve(promise)`), empty array case, first rejection rejects all.

17.12 Implement `Promise.race`

JAVASCRIPT Copy

```
function promiseRace(promises) {
  return new Promise((resolve, reject) => {
    for (const promise of promises) {
      Promise.resolve(promise).then(resolve, reject);
    }
  });
}
```

First to settle (fulfill or reject) wins.

17.13 Implement `Event Emitter`

JAVASCRIPT Copy

```
class EventEmitter {
  constructor() {
    this.events = new Map();
  }

  on(event, listener) {
    if (!this.events.has(event)) this.events.set(event, []);
    this.events.get(event).push(listener);
    return this; // Chainable
  }

  off(event, listener) {
    const listeners = this.events.get(event);
    if (!listeners) return this;
    this.events.set(event, listeners.filter(l => l !== listener && l._orig
    return this;
  }

  emit(event, ...args) {
    const listeners = this.events.get(event);
```

```
    }

    once(event, listener) {
      const wrapper = (...args) => {
        listener(...args);
        this.off(event, wrapper);
      };
      wrapper._original = listener; // So off(event, listener) works
      this.on(event, wrapper);
      return this;
    }

    removeAllListeners(event) {
      if (event) this.events.delete(event);
      else this.events.clear();
      return this;
    }
}

// Test
const emitter = new EventEmitter();
emitter.on('data', (msg) => console.log(`Received: ${msg}`));
emitter.once('connect', () => console.log('Connected!'));
emitter.emit('connect');    // "Connected!"
emitter.emit('connect');    // (nothing - once listener removed)
emitter.emit('data', 'hi'); // "Received: hi"
```

17.14 Implement `getElementsByClassName` (DOM traversal)

JAVASCRIPT

Copy

```
function getElementsByClassName(root, className) {
  const result = [];

  function traverse(node) {
```

```
  }

  traverse(root);
  return result;
}
```

What interviewers check: Recursive DOM traversal, understanding `nodeType`, not including the root if it doesn't match, handling multiple classes on an element.

17.15 Implement `querySelector` (CSS selector matching)

JAVASCRIPT Copy

```
// Simplified — handles tag, .class, #id, and combinations
function querySelector(root, selector) {
  function matches(element, sel) {
    if (sel.startsWith('#')) return element.id === sel.slice(1);
    if (sel.startsWith('.')) return element.classList.contains(sel.slice(1
    return element.tagName.toLowerCase() === sel.toLowerCase();
  }

  function traverse(node) {
    if (node.nodeType !== 1) return null;
    if (matches(node, selector)) return node;
    for (const child of node.children) {
      const found = traverse(child);
      if (found) return found;
    }
    return null;
  }

  return traverse(root);
}
```

JAVASCRIPT Copy

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function(...moreArgs) {
      return curried.apply(this, [...args, ...moreArgs]);
    };
  };
}

// Test
const add = curry((a, b, c) => a + b + c);
add(1)(2)(3);     // 6
add(1, 2)(3);     // 6
add(1)(2, 3);     // 6
add(1, 2, 3);     // 6
```

What interviewers check: `fn.length` gives the expected argument count, partial application works with any combination of arguments.

17.17 Implement `pipe` and `compose`

JAVASCRIPT Copy

```
function pipe(...fns) {
  return function(x) {
    return fns.reduce((acc, fn) => fn(acc), x);
  };
}

function compose(...fns) {
  return function(x) {
    return fns.reduceRight((acc, fn) => fn(acc), x);
  };
```

```
// Test
const double = x => x * 2;
const addOne = x => x + 1;
const square = x => x * x;

pipe(double, addOne, square)(3);    // square(addOne(double(3))) = square(
compose(square, addOne, double)(3); // square(addOne(double(3))) = 49
```

`pipe`: left to right. `compose`: right to left. Both create function pipelines.

17.18 Implement `Object.create`

JAVASCRIPT Copy

```
function objectCreate(proto) {
  if (proto === null || typeof proto === 'object' || typeof proto === 'fun
    function F() {}
    F.prototype = proto;
    return new F();
  }
  throw new TypeError('Object prototype may only be an Object or null');
}

// Test
const parent = { greet() { return `Hello, ${this.name}`; } };
const child = objectCreate(parent);
child.name = 'Ankit';
child.greet(); // "Hello, Ankit" — inherited from parent
```

17.19 Implement `typeof` (comprehensive type checker)

JAVASCRIPT Copy

```
  if (Array.isArray(value)) return 'array';
  if (value instanceof Date) return 'date';
  if (value instanceof RegExp) return 'regexp';
  if (value instanceof Map) return 'map';
  if (value instanceof Set) return 'set';
  if (value instanceof Promise) return 'promise';
  return typeof value; // 'number', 'string', 'boolean', 'undefined', 'fun
}

// More robust version using Object.prototype.toString
function typeOfRobust(value) {
  return Object.prototype.toString.call(value).slice(8, -1).toLowerCase();
}

typeOfRobust(null);            // 'null'
typeOfRobust([1, 2]);          // 'array'
typeOfRobust(new Date());      // 'date'
typeOfRobust(/regex/);         // 'regexp'
typeOfRobust(new Map());       // 'map'
typeOfRobust(Promise.resolve()); // 'promise'
```

17.20 Implement `setInterval` using `setTimeout`

JAVASCRIPT Copy

```
function mySetInterval(callback, delay) {
  let cancelled = false;

  function tick() {
    if (cancelled) return;
    callback();
    setTimeout(tick, delay);
  }

  setTimeout(tick, delay);
```

```
// Return cancel function (like clearInterval)
  return { cancel: () => { cancelled = true; } };
}

// Test
const interval = mySetInterval(() => console.log('tick'), 1000);
// "tick" every second
setTimeout(() => interval.cancel(), 5000); // Stop after 5 seconds
```

What interviewers check: Recursive `setTimeout` (not `setInterval` inside), cancellation mechanism, understanding that `setTimeout` in a recursive pattern provides more control (each tick can have a different delay).

Bonus question interviewers ask: "Why is recursive `setTimeout` sometimes preferred over `setInterval`?"

`setInterval` fires regardless of whether the previous callback has finished. If the callback takes 800ms and the interval is 1000ms, there's only 200ms gap. If it takes 1200ms, callbacks start stacking up. Recursive `setTimeout` guarantees the delay happens AFTER the callback completes.

QUICK CHECK — no marks, just for you.

1. Using the lesson's myMap polyfill, what does this snippet print?

JAVASCRIPT Copy

```
Array.prototype.myMap = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) {
      result[i] = callback.call(thisArg, this[i], i, this);
    }
  }
  return result;
};

const out = [1, , 3].myMap(x => x * 2);
console.log(out.length);
```

- 3
  true
  [2,null,6]
- 2
  false
  [2,6]
- 3
  false
  [2,null,6]
- 3
  false
  [2,undefined,6]

Check answer

2. Given the lesson's EventEmitter, what does this snippet print?

JAVASCRIPT Copy

```
class EventEmitter {
  constructor() {
    this.events = new Map();
  }
  on(event, listener) {
    if (!this.events.has(event)) this.events.set(event, []);
    this.events.get(event).push(listener);
    return this;
  }
  off(event, listener) {
    const listeners = this.events.get(event);
    if (!listeners) return this;
    this.events.set(event, listeners.filter(l => l !== listener && l._origir
    return this;
  }
  emit(event, ...args) {
    const listeners = this.events.get(event);
```

```
  }
  once(event, listener) {
    const wrapper = (...args) => {
      listener(...args);
      this.off(event, wrapper);
    };
    wrapper._original = listener;
    this.on(event, wrapper);
    return this;
  }
}

const emitter = new EventEmitter();
emitter.once('ping', () => console.log('pong'));
console.log(emitter.emit('ping'));
console.log(emitter.emit('ping'));
```

- pong
  true
  true
- pong
  true
  false
- pong
  pong
  true
  true
- pong
  true
  pong
  true

Check answer

3. You need a handler for a window scroll event that must keep running at a steady rate while the user scrolls, without firing on every single scroll tick. According to the lesson, which utility

fits, and why?

- debounce, because it collapses the whole scroll stream into a single call once scrolling stops
- Either one, since debounce and throttle produce the same call pattern and differ only in argument order
- throttle, because it fires at most once per interval and so keeps firing regularly during a continuous stream of events
- debounce with leading set to true, because that is the only way to get a call during a continuous stream of events

Check answer

‹ Previous
Custom hooks and rules

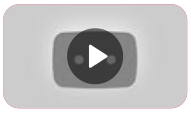
Next ›
JS Problems SDE2

LEARN MORE

ARTICLES & DOCS

JS interview questions (repo)
github.com

VIDEOS



Array & string interview questions
YouTube



9 array methods + practice
YouTube



String & array problems step by step
YouTube

Hand-picked references for this lesson.